Module 13 — LLM CI/CD & Release Engineering

> **Goal:** After this chapter you can (a) define the deployable unit of an LLM application and version it as one bundle, (b) design CI with eval gates that fit a token budget and a noise floor, and (c) ship, experiment on, upgrade and roll back models, prompts and indexes with measurable guard metrics.

Table of Contents

- 13.1 The Release Bundle: Models, Prompts, Indexes, Guardrails and Tool Schemas as One Unit
- 13.2 CI for LLM Applications: Test Pyramid, Eval Gates and Cost Control
- 13.3 Progressive Delivery: Shadow, Canary, Blue-Green and Feature Flags
- 13.4 Model and Provider Upgrades: The Deprecation Playbook
- 13.5 Online Experimentation: A/B Tests, Interleaving and Sequential Decisions
- 13.6 Rollback, Migrations and Kill Switches: GitOps for LLM Systems

13.1 The Release Bundle: Models, Prompts, Indexes, Guardrails and Tool Schemas as One Unit

Conceptual Deep Dive

An LLM application release is a **release bundle**: base model snapshot or adapter, tokeniser and chat template, prompt versions, index or collection version with its embedding model, guardrail policy, tool schemas and configuration. Changing one component without the others produces combinations nobody tested: a new prompt written for a new output schema, served with an old tool schema.

The bundle **manifest** lists every component with a version and a content hash, and the bundle id is a hash of the manifest. A **lockfile** concept applies: the manifest pins exact versions, never ranges. The manifest is the unit that is evaluated, promoted and rolled back (it extends the release-manifest linter in Module 12 section 3.3).

Registries hold each artefact type: a model registry (weights, tokeniser, quantisation metadata), a prompt registry (Module 03 section 3.2), an index registry (collection, embedding model, chunker version) and a policy registry (guardrails). Promotion moves **aliases** (dev, staging, prod) to a bundle id; it never copies files, so what was evaluated is byte-identical to what runs.

Versioning policy: prompts use semantic versions where a MAJOR bump means a change to the output schema or behaviour contract; adapters pin the base-model hash; indexes pin the embedding model id. Load-time **compatibility checks** enforce the matrix: adapter versus base, embedding model versus index, tool schema versus prompt, tokeniser versus chat template. Fail at startup, not at the first user request.

Provenance: record who built the bundle, from which commit, and which evaluation report approved it. Signing artefacts (for example with a Sigstore-style tool, verify current options) turns provenance into something a deploy step can enforce.

Component	Pin by	Compatibility rule
Model / adapter	snapshot id, base hash	adapter.base_hash == served base
Prompt	semver + content hash	MAJOR matches output parser
Index	snapshot id, embedder id	query embedder == index embedder
Guardrail policy	version + hash	evaluated with the bundle
Tool schemas	version	prompt references valid tools

Edge cases. A hotfix to a prompt alone still produces a new bundle id, so traces and rollbacks stay exact. An index rebuilt nightly means the bundle references an index snapshot id, never "latest"; otherwise the evaluated and served indexes differ.

Cross-Connection Scenarios

- Prompt registry and review: Module 03 section 3.2; model metadata: Module 04.
- Index aliases and embedding pinning: Module 06.
- Guardrail policy versions: Module 08; manifest linter: Module 12 section 3.3.
- Maturity level L2 requires this bundle: 1.6.

Production Anti-Patterns

- "latest" tags anywhere in production: the running system cannot be reproduced or bisected.
- Promoting a prompt to prod by editing an environment variable with no review: untested, unaudited behaviour change.
- A bundle that omits the guardrail policy: safety changes ship silently outside the gates.

Minimal Working Code

The first block builds a bundle manifest with content hashes and validates required components and non-floating versions.

```
import hashlib
import json

REQUIRED = ["model", "tokenizer", "prompt", "index", "guardr
ail_policy", "tool_schemas", "config"]
FLOATING = {"latest", "main", "head", ""}

def sha(obj) -> str:
    return hashlib.sha256(json.dumps(obj, sort_keys=True).encode()).hexdigest()

def build_manifest(components: dict) -> dict:
    comps = {k: {"version": v["version"], "hash": sha(v["content"
])} for k, v in components.items()}
    return {"bundle_id": "b-" + sha(comps)[:12], "components": comps}

def validate(m: dict):
    errs = [f"missing {r}" for r in REQUIRED if r not in m["components"]]
    errs += [f"floating version: {k}" for k, c in m["components"].items()
             if str(c["version"]).lower() in FLOATING]
    return errs

comps = {k: {"version": "1.0.0", "content": {"name": k}} for k in
 REQUIRED}
m1 = build_manifest(comps)
comps["prompt"]["content"] = {"name": "prompt", "text":
 "hotfix"}
m2 = build_manifest(comps)
comps["index"]["version"] = "latest"
```

```
print(m1["bundle_id"], m2["bundle_id"], validate(build_manifest(comps)))
assert m1["bundle_id"] != m2["bundle_id"] and validate(m1) == []
```

The second block checks compatibility and implements an in-memory alias registry with promote and rollback.

```
def check_compat(info: dict):
    errs = []
    if info["adapter_base_hash"] != info["served_base_hash"]:
        errs.append("adapter/base mismatch")
    if info["index_embedder"] != info["query_embedder"]:
        errs.append("index/query embedder mismatch")
    return errs

class AliasRegistry:
    def __init__(self):
        self.alias, self.history = {}, {}

    def promote(self, alias: str, bundle_id: str):
        if alias in self.alias:
            self.history.setdefault(alias, []).append(self.alias[alias])
        self.alias[alias] = bundle_id

    def rollback(self, alias: str):
        self.alias[alias] = self.history[alias].pop()
        return self.alias[alias]

print(check_compat({"adapter_base_hash": "a1", "served_base_hash": "a2
",
                    "index_embedder": "emb-v1", "query_embedder": "emb
-v1"}))
reg = AliasRegistry()
reg.promote("prod", "b-111")
reg.promote("prod", "b-222")
assert reg.rollback("prod") == "b-111" and reg.alias["prod"] == "b-111
"
```

Interview Angle

Q: What exactly is the deployable unit of an LLM application and why? **A:** The bundle: model or adapter, tokeniser and template, prompts, index and embedder, guardrail policy, tool schemas and config, identified by a content hash. Evaluating anything smaller tests combinations that never ship. Trade-off: heavier release process versus reproducibility. Check: given a trace, you can reconstruct the exact bundle and rerun it to get the same behaviour within the noise band.

Q: A rollback restored the old prompt but quality is still bad. What did you forget? **A:** Components outside the prompt: the index (re-embedded with a new model or changed chunking), guardrail policy, tool schema, the cache holding answers from the bad version, or an adapter. Cure: roll back the bundle alias as a whole, namespace caches by bundle id, and run the golden set after rollback to verify recovery.

Q: How do you guarantee that what you evaluated is what you shipped? **A:** Evaluate the bundle id, promote by alias without rebuilding, and have the serving layer verify hashes at start-up and log the bundle id in every trace. A deploy step refuses a bundle with no passing eval report attached. Check by sampling production traces and matching their bundle ids to eval reports.

13.2 CI for LLM Applications: Test Pyramid, Eval Gates and Cost Control

Conceptual Deep Dive

The LLM test pyramid, bottom to top: **unit tests** of pure code (parsers, validators, chunkers); **contract tests** (tool and output schemas); **deterministic replay tests** from recorded calls; a **small smoke eval** on every pull request; the **full eval** nightly and pre-release; and **load or performance tests** before infrastructure changes. Most of the code is deterministic and should be tested without a model.

Flakiness. Temperature 0 is not deterministic across batch compositions, kernels and provider updates, so measure run-to-run noise by repeating the suite several times, and set gates above the noise floor. Quarantine flaky items (tag, track, fix or retire) instead of deleting them or re-running until green.

Record and replay (cassettes). Persist LLM and judge calls keyed by a hash of (model snapshot, prompt, parameters). Replay makes tests fast, free and reproducible. In CI use *strict mode*, which fails on a cache miss, so a prompt edit cannot silently trigger paid live calls; refreshing cassettes is an explicit, reviewed job.

Selective execution and budgets. Run only the suites affected by changed artefacts (a prompt change runs prompt evals, a chunker change runs retrieval evals). Budget CI by tokens and dollars and fail the build when the budget is exceeded.

Gate design. Use non-inferiority with a paired bootstrap on the golden set plus slice guards (Module 07 section 3.8), not a single mean. Publish a markdown summary on the pull request: delta, confidence interval, margin, slices, cost.

Secrets hygiene. CI never sees production keys. Use scoped test keys with spend caps.

Layer	Runs	Model calls	Catches
Unit / contract	every commit	none	logic and schema bugs
Replay	every PR	none (cassette)	regressions in glue code
Smoke eval	every PR	small, budgeted	gross quality drops
Full eval	nightly, pre-release	full	slice and tail regressions
Load test	before infra change	engine only	latency and capacity

Edge cases. Fork PRs have no access to secrets: run replay-only tests and require a maintainer-triggered job for live evals. If the judge model is upgraded mid-week, baselines are invalidated; version the judge and re-baseline.

Cross-Connection Scenarios

- Gates and judge calibration: Module 07 section 3.8 and Module 07 judge sections.
- Online follow-up of offline gates: 13.3 and 13.5.
- Release bundle that CI evaluates: 13.1.
- Needle and compaction regression tests: 3.6.

Production Anti-Patterns

- Gating on a single aggregate mean with no confidence interval: noise decides merges.
- Calling the live model in every unit test: slow, expensive, flaky pipelines that developers bypass.
- Re-running evals until green: the gate stops measuring anything.

Minimal Working Code

The first block wraps an LLM function with record/replay persisted to JSON and a strict mode that fails on cache misses.

```
import hashlib
import json
import os
import tempfile

class Cassette:
    def __init__(self, fn, path, strict=False):
        self.fn, self.path, self.strict = fn, path, strict
        self.data = {}
        if os.path.exists(path):
            with open(path) as f:
                self.data = json.load(f)

    def __call__(self, prompt, **params):
        key = hashlib.sha256(json.dumps([prompt, params], sort_keys=True).encode()).hexdigest()
        if key not in self.data:
            if self.strict:
                raise KeyError("cassette miss in strict mode")
            self.data[key] = self.fn(prompt, **params)
```

```
            with open(self.path, "w") as f:
                json.dump(self.data, f)
        return self.data[key]

calls = []
def fake_llm(prompt, **params):
    calls.append(prompt)
    return prompt.upper()

path = os.path.join(tempfile.mkdtemp(), "cassette.json")
rec = Cassette(fake_llm, path)
assert rec("hi", model="MODEL_ID") == "HI"
strict = Cassette(fake_llm, path, strict=True)
assert strict("hi", model="MODEL_ID") == "HI" and len(calls) == 1
try:
    strict("new prompt", model="MODEL_ID")
except KeyError as e:
    print("strict mode blocked a live call:", e)
```

The second block is a gate comparing candidate and baseline per-item scores with a paired bootstrap, a non-inferiority margin, an exit code and a markdown report.

```
import numpy as np

def gate(base, cand, margin=0.02, n_boot=2000, seed=0):
    d = np.asarray(cand, float) - np.asarray(base, float)
    rng = np.random.default_rng(seed)
    means = np.array([d[rng.integers(0, len(d), len(d))].mean() for _ in range(n_boot)])
    lo, hi = np.percentile(means, [2.5, 97.5])
    ok = lo > -margin
    md = (f"| metric | delta | 95% CI | margin | verdict |\n|---|---|---|---|---|\n"
          f"| score | {d.mean():+.3f} | [{lo:+.3f}, {hi:+.3f}] | -{margin} | {'PASS' if ok else 'FAI
L'} |")
    return (0 if ok else 1), md

rng = np.random.default_rng(3)
base = (rng.random(300) < 0.80).astype(float)
cand_ok = np.where(rng.random(300) < 0.97, base, 1 - base)
cand_bad = np.where(rng.random(300) < 0.80, base, 0.0)
for name, c in (("candidate A", cand_ok), ("candidate B", cand_bad)):
    code, md = gate(base, c)
    print(name, "exit", code)
    print(md)
assert gate(base, cand_bad)[0] == 1
```

Interview Angle

Q: Design CI for a RAG application with a 300-item golden set and a $20 per-run budget. **A:** Every PR: unit, contract and replay tests free of model calls, plus a 60-item stratified smoke eval with cassettes. Nightly: the full 300 items with 3 repeats to measure noise. Gate by paired bootstrap non-inferiority with slice guards; fail on budget overrun. Estimate cost as items times tokens times price before running. Check that gate decisions are stable across repeated runs.

Q: Your eval flips between pass and fail on identical code. What do you do? **A:** Measure the noise: repeat the suite, compute the standard deviation of the score, and raise the gate margin or sample size above it. Identify flaky items by per-item variance and quarantine them; fix sources of nondeterminism (seeds, retrieval ties, judge sampling). Never re-run until green. Trade-off: larger runs cost more; check that identical code passes at the target rate.

Q: Which tests belong on every PR and which nightly, and why? **A:** PR: anything fast, free or affected by the diff (unit, contract, replay, small smoke eval) so feedback arrives in minutes. Nightly and pre-release: the full golden set, long-context and safety slices, judge-based evals and load tests, because they are costly and need statistical power. Check PR time under about ten minutes and nightly cost within budget.

13.3 Progressive Delivery: Shadow, Canary, Blue-Green and Feature Flags

Conceptual Deep Dive

Shadow traffic mirrors real requests to the candidate, discards its response and compares outputs, latency and cost against the baseline. Safety rule: shadow must never execute side-effecting tools (emails, refunds, writes); stub them or run in dry-run mode, and keep the candidate's data access read-only.

A **canary** routes a small, *sticky* fraction of users to the candidate: hash of user id plus a salt, so a conversation does not flip between versions mid-session. Use a ramp schedule (for example 1%, 5%, 25%, 100%) with a bake time at each step long enough to see slow metrics. **Blue-green** keeps two full environments (model pools, index versions) and flips an alias, giving instant rollback; cache namespaces include the bundle id so answers from one version never serve the other.

Automatic analysis compares guard metrics between arms: error rate, p95 time to first token and inter-token latency, refusal rate, format validity, judge score on sampled traces, cost per request and thumbs-down rate. The decision logic is PROMOTE, HOLD or ROLLBACK, with explicit reasons and a minimum sample size per metric.

Ring deployment exposes the change progressively by tenant risk (internal users, beta customers, everyone). **Feature flags** gate prompts and tools independently of deploys, with owners and expiry dates.

Mechanism	Risk exposed	Cost	Best for
Shadow	none to users	doubled compute	model or prompt behavioural diff
Canary	small user fraction	low	most releases
Blue-green	none until flip	duplicate capacity	model pool and index swaps
Feature flag	per feature/tenant	low	tools, new behaviours

Stateful gotchas: in-flight conversations, long agent runs that span a deploy (pin the run to the bundle it started with) and cached answers from the old version.

Edge cases. Low-traffic features cannot reach statistical power at a 5% canary; use longer bake time, a larger cohort for internal users, or synthetic probes. Quality metrics lag (human feedback arrives days later): gate on judge-score proxies and delayed confirmation, and keep the previous bundle ready for rollback for that period.

Cross-Connection Scenarios

- Bundle ids and aliases: 13.1 and 13.6.
- Metrics and tracing for guard signals: Module 10.
- Statistical decisions beyond canaries: 13.5.
- Agent runs that span deploys: Module 09 (durable execution).

Production Anti-Patterns

- Canary by request instead of by user: one user sees both versions in a conversation and results are contaminated.
- Shadow mode that actually sends emails or refunds: the "safe" test causes real side effects.
- Promotion decided on latency alone: a quality regression passes because the system is fast.

Minimal Working Code

The first block is a deterministic splitter with sticky assignment and a ramp schedule.

```
import hashlib

def bucket(user_id: str, salt: str) -> int:
    return int(hashlib.sha256(f"{user_id}:{salt}".encode()).hexdigest()[:8], 16) % 10_000

def in_canary(user_id: str, pct: float, salt: str = "rel-42") -> bool:
    return bucket(user_id, salt) < pct * 100          # pct in percent
```

```
RAMP = [(1, "30m"), (5, "2h"), (25, "6h"), (100, "-")]  # (percen
t, bake time)

users = [f"user{i}" for i in range(20_000)]
share = sum(in_canary(u, 5) for u in users) / len(users)
print("canary share at 5%:", round(share, 3))
assert all(in_canary(u, 5) == in_canary(u, 5) for u in users[:100])          # sticky
assert abs(share - 0.05) < 0.01
assert {u for u in users if in_canary(u, 1)} <= {u for u in users if in_canary(u, 5)}   # ramp is nest
ed
```

The second block compares baseline and canary counters using Wilson bounds and returns PROMOTE, HOLD or ROLLBACK with reasons.

```
import math

def wilson(k, n, z=1.96):
    if n == 0:
        return 0.0, 1.0
    p, d = k / n, 1 + z * z / n
    c = p + z * z / (2 * n)
    w = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n))
    return (c - w) / d, (c + w) / d

def analyse(base, can, min_n=500, lat_ratio=1.3):
    reasons = []
    if can["n"] < min_n:
        return "HOLD", ["insufficient canary traffic"]
    for metric in ("errors", "refusals", "invalid_json", "thumbs_down&
quot;):
        c_lo, _ = wilson(can[metric], can["n"])
        _, b_hi = wilson(base[metric], base["n"])
        if c_lo > b_hi:
            reasons.append(f"{metric} significantly worse")
    if can["p95_ms"] > lat_ratio * base["p95_ms"]:
        reasons.append("p95 latency regression")
    return ("ROLLBACK" if reasons else "PROMOTE"), reasons

base = dict(n=20000, errors=40, refusals=300, invalid_json=60, thumbs_down=400, p95_ms=900)
good = dict(n=1000, errors=2, refusals=14, invalid_json=3, thumbs_down=21, p95_ms=950)
bad = dict(n=1000, errors=2, refusals=14, invalid_json=40, thumbs_down=21, p95_ms=950)
print(analyse(base, good), analyse(base, bad), analyse(base, dict(good, n=100)))
assert analyse(base, good)[0] == "PROMOTE" and analyse(base, bad)[0] == "ROLLBACK"
```

Interview Angle

Q: Design a canary for a new prompt version of a customer-facing assistant. Which metrics gate it and how long do you bake? **A:** Sticky by user, ramp 1/5/25/100%. Guards: error rate, p95 TTFT, refusal rate, JSON validity, sampled judge score, cost per request, thumbs-down rate, with Wilson-bound comparisons. Bake each step long enough to cover a daily traffic cycle and the minimum sample size per metric. Trade-off: slower rollout versus risk; check that the analyser would have caught a seeded regression in a rehearsal.

Q: How do you shadow-test an agent that has side effects? **A:** Run the candidate against mirrored inputs with tool calls replaced by stubs or dry-run endpoints, compare the planned tool calls and final answers to the baseline's, and never share write credentials. Limits: stubs cannot reproduce tool responses that depend on prior effects, so complement with a small canary for stateful flows.

Q: Canary shows no difference in 2 hours but quality complaints appear next day. What was missing? **A:** Bake time and metrics. Slow signals (delayed feedback, long conversations, rare intents, specific tenants) were not observable in 2 hours or were averaged away. Add slice guards, longer bake with ring rollout, judge-scored sampling of long sessions and a delayed confirmation step before removing the old bundle.

13.4 Model and Provider Upgrades: The Deprecation Playbook

Conceptual Deep Dive

Treat an upgrade as a release with an **evidence pack**: golden-set results with confidence intervals by slice, safety and refusal behaviour, tool-call and JSON validity, latency and cost, and long-context behaviour. "The vendor says it is better" is not evidence.

What changes silently: the **tokeniser** (the same text costs a different number of tokens, so bills and context budgets shift), the **chat template** and stop tokens, formatting habits (markdown, lists), refusal style, verbosity, and adherence to tool-call schemas. A new model that is better on average can be worse for one language, tenant or task.

Parallel run. Shadow the candidate on production prompts (13.3) and produce a behavioural diff report before any traffic moves: length distribution, JSON validity, refusal markers, latency percentiles, token counts, and a judged pairwise comparison on a sample.

Re-tuning and fallbacks. Prompts tuned for one model rarely transfer perfectly; re-tune per model (3.7) and re-validate the fallback chain against the same gates (1.8). For hosted providers, review the contract and SLA, data-use terms and regional availability for the new model.

Self-hosted upgrades. Pin engine and driver versions; kernel changes alter numerics (4.7), so rerun the compression and quality harness even when weights are unchanged.

Communication and rollback. Announce the date, the expected behavioural changes and the rollback window. Keep the old version running through a bake period after cutover and remove it only when the rollback window has passed and nothing depends on it.

Dimension	Check	Typical surprise
Quality	slice CIs	one language regresses
Cost	tokens per task	tokeniser inflates input
Latency	p95/p99	reasoning mode adds tail
Format	JSON/tool validity	stricter or looser schemas
Safety	refusal and red-team set	different refusal boundaries

Edge cases. A provider updates a floating alias without notice: detect it through the response model id in traces and a daily canary golden set (Module 10), and pin dated snapshots (1.8). A new model improves the average but hurts one language or tenant: ship only with a routing exception or hold the migration for that slice.

Cross-Connection Scenarios

- Selection, inventory and sunset calendar: 1.8.
- Prompt re-optimisation per model: 3.7; reasoning variants: 3.8.
- Numerics under engine change: 4.7.
- Silent-update detection and dashboards: Module 10.

Production Anti-Patterns

- Upgrading because the vendor blog says it is better: no evidence for your task, slices or cost.
- Skipping prompt re-tuning and blaming the model: the new model is measured with the old model's prompt.
- Removing the old version the same day: no rollback path when a slow regression appears.

Minimal Working Code

The first block produces a behavioural diff report between two runs on the same prompts.

```
import json
import statistics

def pct(xs, p):
    xs = sorted(xs)
    return xs[min(len(xs) - 1, int(p * len(xs)))]
```

```
def valid_json(s):
    try:
        json.loads(s)
        return True
    except ValueError:
        return False

def summarise(run):
    return {"len_p50": statistics.median(len(r["text"]) for r in run),
            "json_valid": sum(valid_json(r["text"]) for r in run) / len(run),
            "refusal": sum("cannot help" in r["text"].lower() for r in run)
 / len(run),
            "lat_p95": pct([r["latency"] for r in run], 0.95),
            "tokens_mean": statistics.mean(r["tokens"] for r in run)}

old = [{"text": '{"a": 1}', "latency": 0.5 + i / 100, "tokens": 2
0} for i in range(50)]
new = [{"text": '{"a": 1}' if i % 5 else "I cannot help with that", "l
atency": 0.6 + i / 100,
        "tokens": 26} for i in range(50)]
a, b = summarise(old), summarise(new)
for k in a:
    print(f"{k:12} {a[k]:8.2f} -> {b[k]:8.2f}")
assert b["json_valid"] < a["json_valid"] and b["refusal"] > a["r
efusal"]
```

The second block estimates the monthly cost impact of a tokeniser change using characters-per-token parameters you supply.

```
import math

def monthly_cost(samples, cpt_in, cpt_out, requests, price_in, price_out):
    t_in = sum(math.ceil(len(p) / cpt_in) for p, _ in samples) / len(samples)
    t_out = sum(math.ceil(len(o) / cpt_out) for _, o in samples) / len(samples)
    return requests * (t_in * price_in + t_out * price_out) / 1e6, t_in, t_out

samples = [("Summarise the attached incident report for the on-call engineer. " * 5, "Shor
t summary. " * 8)
           for _ in range(10)]
# cpt values are assumptions you measure per model; prices are placeholders (perishable)
old = monthly_cost(samples, cpt_in=4.0, cpt_out=4.0, requests=2_000_000, price_in=1.0, price_out=4.0)
new = monthly_cost(samples, cpt_in=3.4, cpt_out=3.4, requests=2_000_000, price_in=1.0, price_out=4.0)
print(f"old {old[0]:.0f}  new {new[0]:.0f}  change {100 * (new[0] / old[0] - 1):+.1f}%")
assert new[0] > old[0]
```

Interview Angle

Q: Your provider deprecates a model in 60 days. Give the migration plan with dates and gates. **A:** Days 0-7: inventory dependants, shortlist candidates, pick a pinned snapshot. Days 8-21: run the eval and shadow diff, re-tune prompts. Days 22-35: gate on the evidence pack (slice CIs, safety, JSON validity, cost), then canary. Days 36-50: ramp and bake, fallback re-validation. Days 51-60: cut over, keep rollback. Trade-off: speed versus evidence; keep a pre-qualified second candidate in case the deadline moves.

Q: What do you compare in a behavioural diff before switching models? **A:** On the same prompts: output length distribution, format and JSON validity, refusal and safety marker rates, tool-call adherence, latency percentiles, token counts and cost, plus a judged pairwise preference on a sample, all by slice. Large unexplained shifts get a human review. Check that differences are outside the measured run-to-run noise before attributing them to the model.

Q: Why can a "better" model increase your bill? **A:** A different tokeniser can turn the same text into more tokens; a more verbose style raises output tokens; reasoning variants add hidden reasoning tokens; a larger model has a higher price per token; retries rise if format validity drops. Compute cost per successful task on your traffic sample, not per-token list prices, before migrating.

13.5 Online Experimentation: A/B Tests, Interleaving and Sequential Decisions

Conceptual Deep Dive

Randomise by **user or session**, not request, so one person does not see both variants and learning effects stay within an arm. Choose one primary metric, a few guardrails (cost, latency, safety, refusal rate) and diagnostics. Decide the minimum detectable effect (MDE) and compute the required sample size before starting; underpowered tests produce noise that looks like findings.

Run a **sample ratio mismatch (SRM)** check before reading any result: a chi-square test of observed arm counts against the intended split. A failed SRM means assignment, logging or filtering is broken (bots, crashes in one arm, cache routing) and the result is invalid until explained.

Peeking inflates false positives: checking a fixed-horizon test daily and stopping at the first $p < 0.05$ yields far more than 5% false positives. Use a fixed horizon, alpha-spending boundaries (Pocock, O'Brien-Fleming style) or always-valid sequential methods (mSPRT concept).

LLM-specific issues: sampling variance (the same prompt gives different answers); long feedback delay; judge-score proxies must be validated against human or user outcomes (14.6); novelty effects; and shared GPU interference between arms (one arm's load slows the other, so isolate or account for it).

Interleaving mixes results from two rankers in one list and credits clicks, giving much higher sensitivity for retrieval and ranking changes with fewer users. **Bandits** can route among models adaptively but need caution: non-stationarity, delayed reward and the loss of clean inference. Close each experiment with a decision record: ship, iterate or kill, and what was learned.

Method	Use when	Caveat
Fixed-horizon A/B	clear primary metric	no peeking
Sequential / alpha-spending	need early stopping	wider boundaries
Interleaving	ranking or retrieval	needs comparable result lists
Bandit	many cheap variants	biased estimates, drift

Edge cases. Small tenants lack power: pool across tenants or run longer; segment analyses need multiplicity control (14.5). Cache sharing between arms contaminates results (arm B reads answers cached by arm A): namespace caches by arm.

Cross-Connection Scenarios

- Offline gates that precede an experiment: 13.2.
- Canary guard metrics versus experiment metrics: 13.3.
- Judge validation and multiplicity control: 14.5 and 14.6.
- Retrieval ranking changes: Module 06.

Production Anti-Patterns

- Stopping the test the first time $p < 0.05$: false positives are several times the nominal rate.
- Randomising by request so a user sees both variants: carry-over effects and unclear attribution.
- Optimising a proxy judge score that does not track user outcomes: the metric improves while users get no benefit.

Minimal Working Code

The first block runs an SRM chi-square test and computes sample size for two proportions.

```
import math
from statistics import NormalDist

def srm_p(n_a, n_b, expected_a=0.5):
    n = n_a + n_b
    ea, eb = n * expected_a, n * (1 - expected_a)
    chi2 = (n_a - ea) ** 2 / ea + (n_b - eb) ** 2 / eb
    return math.erfc(math.sqrt(chi2 / 2))          # df = 1

def sample_size(p1, mde, alpha=0.05, power=0.8):
    p2, nd = p1 + mde, NormalDist()
    za, zb = nd.inv_cdf(1 - alpha / 2), nd.inv_cdf(power)
    pbar = (p1 + p2) / 2
```

```
    num = (za * math.sqrt(2 * pbar * (1 - pbar)) + zb * math.sqrt(p1 * (1 - p1) + p2 * (1 - p2))) ** 2
    return math.ceil(num / mde ** 2)

print("SRM p-values:", srm_p(5000, 5000), srm_p(5000, 5400))
print("n per arm:", sample_size(0.10, 0.02))
assert srm_p(5000, 5400) < 0.01 < srm_p(5000, 5050)
assert 3500 < sample_size(0.10, 0.02) < 4200
```

The second block simulates A/A tests to show false-positive inflation from peeking and a conservative constant boundary that restores the nominal rate.

```
import numpy as np

def fp_rates(sims=20_000, looks=5, per_look=400, p=0.1, seed=0):
    rng = np.random.default_rng(seed)
    a = rng.binomial(per_look, p, (sims, looks)).cumsum(1)
    b = rng.binomial(per_look, p, (sims, looks)).cumsum(1)
    n = per_look * np.arange(1, looks + 1)
    pbar = (a + b) / (2 * n)
    z = np.abs(a - b) / n / np.sqrt(np.maximum(pbar * (1 - pbar) * 2 / n, 1e-12))
    peek = (z > 1.96).any(1).mean()               # stop at first "significant" look
    fixed = (z[:, -1] > 1.96).mean()              # single look at the end
    pocock = (z > 2.413).any(1).mean()            # constant boundary for 5 looks (Pocock, alpha 0.0
5)
    return peek, fixed, pocock

peek, fixed, pocock = fp_rates()
print(f"peeking {peek:.3f}  fixed {fixed:.3f}  pocock {pocock:.3f}")
assert peek > 0.09 and 0.035 < fixed < 0.065 and 0.035 < pocock < 0.065
```

Interview Angle

Q: How would you A/B test a prompt change for an assistant with 50k weekly users? **A:** Randomise by user with sticky assignment. Primary metric: task success or resolution rate; guardrails: cost, p95 latency, refusal and safety rates. Compute sample size from baseline and MDE; run a full weekly cycle; check SRM first; analyse with a pre-registered method. Trade-off: duration versus power. Before launch, pass the offline gate (13.2). Decision record at the end.

Q: SRM check fails: what do you do? **A:** Do not read the result. Investigate assignment and logging: bots filtered in one arm, a crash or timeout in a variant, redirect or cache routing differences, a late-starting arm, user-id changes. Fix the cause and rerun; if the cause is a real treatment effect on logging (variant errors out), that is itself a finding. Check that SRM p-value is healthy on the rerun.

Q: When is interleaving better than A/B for a RAG ranking change? **A:** When the change only reorders or swaps retrieved items and users give click-like preference signals: interleaving uses each user as their own control and needs far fewer users. It cannot measure end-to-end effects such as generation quality or latency, and is awkward when results have no comparable lists, so confirm winners with a small A/B.

13.6 Rollback, Migrations and Kill Switches: GitOps for LLM Systems

Conceptual Deep Dive

Rollback means flipping an alias to a previous immutable bundle (13.1), not rebuilding artefacts. Rehearse it in a game day and time it; an untested rollback fails when needed.

Expand-contract migrations apply to schemas, indexes and embeddings: *expand* by adding the new alongside the old; dual-read or shadow-compare; *switch* traffic; *contract* by removing the old only later. Tool schemas must stay backward compatible while agents mid-run still use them. Changing an embedding model without downtime means building a second index, comparing retrieval quality, flipping the alias and keeping the old index until the rollback window closes.

Kill switches and feature flags need defined precedence (global over tenant over feature), safe defaults, and TTL-cached evaluation so the flag service is not on the hot path. What they control in an LLM system: disable a tool, force a smaller model, return a static fallback, turn off a feature, or stop an agent from taking writes.

GitOps. Desired state lives in Git: manifests (Helm values, Argo CD or Flux sync), image digests rather than tags, drift detection, and policy-as-code checks (no :latest, resource limits set, GPU requests present). Change management adds approvals, freeze windows, an audit trail and blast-radius limits (rollout by ring).

Data that cannot roll back needs plans: fine-tuned adapters trained on bad data, polluted indexes and cached answers. Maintain restore points: previous index snapshots, adapter versions and cache flush procedures.

Control	Purpose	Failure mode
Alias rollback	revert behaviour	stale caches, state outside bundle
Expand-contract	change shape safely	long dual-run cost
Kill switch	immediate off	switch service unavailable
GitOps drift check	detect manual changes	alerts ignored

Edge cases. A rollback blocked by an irreversible schema change is exactly what expand-contract prevents. If the kill-switch service is down, the fail-safe default must be defined in advance per feature (for example tools off, chat on), and the last known configuration should be cached.

Cross-Connection Scenarios

- Immutable bundles and aliases: 13.1; canary and ring rollouts: 13.3.
- Index alias swaps and embedding migrations: Module 06.
- Agent write controls and approvals: Module 09.
- Incident response and on-call: Module 10.

Production Anti-Patterns

- A rollback procedure never exercised: it fails under pressure because credentials, permissions or state were missing.
- Flags with no owner and no expiry: the codebase accumulates untested flag combinations.
- Using :latest image tags in Compose or Helm values: a restart changes production without a commit.

Minimal Working Code

The first block is an alias registry with atomic promote, rollback, history and an audit log.

```
import threading
import time

class Registry:
    def __init__(self):
        self.alias, self.history, self.audit = {}, {}, []
        self._lock = threading.Lock()

    def _log(self, action, alias, frm, to, who):
        self.audit.append({"ts": time.time(), "action": action, "alias": al
ias,
                           "from": frm, "to": to, "who": who})

    def promote(self, alias, bundle_id, who="ci"):
        with self._lock:
            prev = self.alias.get(alias)
            if prev is not None:
                self.history.setdefault(alias, []).append(prev)
            self.alias[alias] = bundle_id
            self._log("promote", alias, prev, bundle_id, who)

    def rollback(self, alias, who="oncall"):
        with self._lock:
            prev = self.history[alias].pop()
            cur, self.alias[alias] = self.alias[alias], prev
            self._log("rollback", alias, cur, prev, who)
            return prev
```

```
r = Registry()
for b in ("b-001", "b-002", "b-003"):
    r.promote("prod", b)
assert r.rollback("prod") == "b-002" and r.alias["prod"] == "b-002&quo
t;
assert r.rollback("prod") == "b-001"
print([(e["action"], e["from"], e["to"]) for e in r.audit])
```

The second block evaluates kill switches with precedence, a TTL cache, stale-config reuse and a fail-safe default.

```
class Switches:
    def __init__(self, fetch, clock, ttl=30, fail_safe=None):
        self.fetch, self.clock, self.ttl = fetch, clock, ttl
        self.fail_safe = fail_safe or {}
        self.cfg, self.ts = None, -1e9

    def _config(self):
        now = self.clock()
        if self.cfg is None or now - self.ts >= self.ttl:
            try:
                self.cfg, self.ts = self.fetch(), now
            except Exception:
                pass                                # keep stale config if we have one
        return self.cfg

    def enabled(self, feature, tenant):
        cfg = self._config()
        if cfg is None:
            return self.fail_safe.get(feature, False)
        for level in (cfg.get("global", {}), cfg.get("tenant", {}).get(tenant, {}), c
fg.get("feature", {})):
            if feature in level:
                return level[feature]               # first explicit entry wins: global > tenant &g
t; feature
        return self.fail_safe.get(feature, False)

now = [0.0]
state = {"cfg": {"global": {"tools": False}, "tenant": {"acm
e": {"chat": False}}, "feature": {"chat": True}}, "up": True
}
def fetch():
    if not state["up"]:
        raise RuntimeError("flag service down")
    return state["cfg"]

sw = Switches(fetch, lambda: now[0], fail_safe={"chat": True})
assert not sw.enabled("tools", "acme") and not sw.enabled("tools", "be
ta")      # global wins
assert not sw.enabled("chat", "acme") and sw.enabled("chat", "beta&quo
t;)           # tenant over feature
state["up"], now[0] = False, 100.0
assert not sw.enabled("tools", "beta")                                  # sta
le config reused
assert Switches(fetch, lambda: 0.0, fail_safe={"chat": True}).enabled("chat", "x
") and not Switches(fetch, lambda: 0.0).enabled("tools", "x")
print("kill-switch rules ok")
```

Interview Angle

Q: Production quality dropped after a release. What is your first 10 minutes? **A:** Confirm with guard metrics and sampled traces that it correlates with the release bundle id; check the canary or kill-switch options; if the signal is clear, flip the alias back (rehearsed, minutes), then verify with the golden set and stabilise caches. Investigate afterwards with the evidence pack. Trade-off: speed versus certainty; rollback is cheap, diagnosis can wait. Check recovery on the same metrics.

Q: How do you change an embedding model without downtime and with the ability to roll back? **A:** Expand-contract: build a second index with the new embedder, run dual retrieval and compare recall and answer quality offline and in shadow, flip the alias with a canary, keep the old index and query-embedding path through the rollback window, then contract. Cost: double storage and embedding compute temporarily. Never mix query and index embedders.

Q: What belongs behind a kill switch in an agent product? **A:** Each side-effecting tool (payments, emails, writes), the agent autonomy level (force human approval), model selection (force a cheaper or safer model), external integrations and the whole feature per tenant. Define fail-safe defaults (tools off when the switch service is unreachable), give each flag an owner and expiry, and test the switches in game days.